Homework 3

Neural networks and reliable experimental evaluation

Total 50 points. Derive and verify neural-network gradients, analyze convolutional architectures, and evaluate classification and regression models using reproducible experiments. Conclusions must be supported by calculations or reported evidence.

Submission

Submit one PDF report and runnable Python code or notebooks with a README and dependency versions. Include all derivations, model specifications, per-run results, summary statistics, plots, and discussion. Code must recreate the splits, training, and evaluation from the supplied files. State the hardware and approximate runtime. A layer table is sufficient; a particular diagramming package is not required.

Question	Topic	Points
1	Gradient propagation and CNN dimensions	10
2	Backpropagation and gradient verification	10
3	Evaluation protocol audit	5
4	DNA composition and sequence-order investigation	15
5	When neural regression helps	10

Shared experimental rules for Questions 4 and 5

Use three prescribed split seeds: 11, 22, and 33. Seeds 44 and 55 are optional; use the same seed set for every compared condition. Share splits across models within each task. Create the test partition first, then split development data into training and validation subsets as specified below. Fit learned preprocessing on training data only.

Before running experiments, record a short hypothesis and experiment plan for each task: your prediction, the factor you will vary, what you will hold fixed, and an outcome that would challenge your prediction. Choose one additional investigation per task, with at most three factor settings including the reference condition. Declare these settings before running the comparison.

For core neural models, declare at most two candidate training configurations. Select separately within each split using only its training and validation data; overlapping partitions make transferring tuned choices between splits inappropriate. For controlled variants, retain the reference model's selected settings within that split, except for the factor being investigated. Select stopping epochs using validation loss, with at most 100 epochs and patience 10. Restore the best checkpoint. Complete all model and experimental choices before inspecting any test results.

Report split, initialization, data-loader, and perturbation seeds; configurations; software versions; hardware; and runtime. Report each run and mean $\pm$ sample standard deviation, plus paired differences on shared splits. Repeated splits overlap: their standard deviation is not a confidence interval. Label unplanned follow-up analyses exploratory and do not use them to claim independent confirmation.

AI coding and assessment

For Questions 4 and 5, AI coding is permitted and encouraged. You may use AI tools to write code, debug errors, suggest experiments, and help explain results. You are responsible for implementation correctness and the validity of your conclusions. Submit runnable code, split seeds, configurations, and per-run results, with your own explanation of the evidence.

Include a brief AI-use statement naming the tools and their main contributions. For each task, document one consequential AI suggestion that you accepted, modified, or rejected and how you checked it. If you did not use AI, discuss one consequential implementation or experimental decision instead. Include one concrete implementation check per task, such as verifying label alignment, train-only scaling, or agreement with a hand-computed metric.

Grades emphasize hypotheses, controls, reproducibility, and interpretation. Neither manually written code volume nor the highest test score is a grading criterion. A well-supported negative or inconclusive result can earn full credit. For Questions 1–3, present your own derivations and reasoning and disclose any assistance.

Question 1 Gradient propagation and CNN dimensions

10 points

A Gradient propagation 4 points

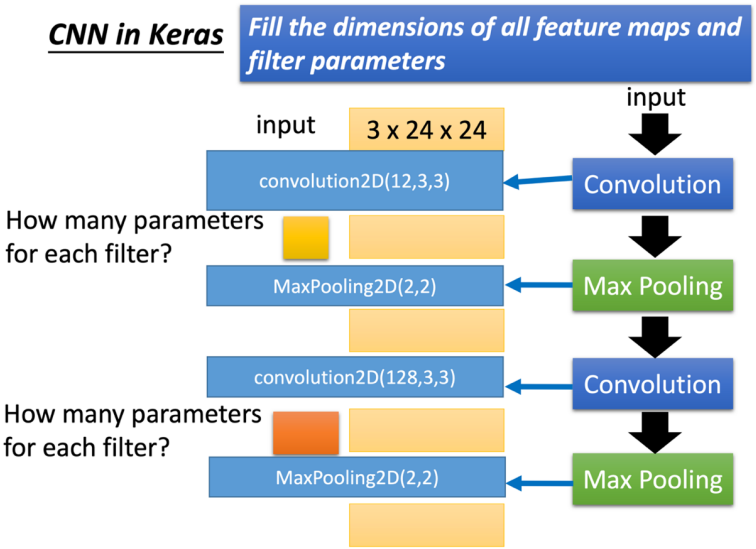
Consider a scalar network $h_0 = x$ and $h_\ell = \sigma(w_\ell h_{\ell-1})$ for $\ell = 1, \ldots, L$, where $\sigma(u) = 1/(1+e^{-u})$ and $|w_\ell| \le 1$. There are no biases.

(i) Use the chain rule to derive an upper bound on $|\partial h_L/\partial x|$ as a function of L. Explain what the bound implies about gradient propagation as depth increases. (2 points)

(ii) Explain why the weight bound matters and why small sigmoid derivatives alone do not prove that every deep network must have vanishing gradients. For a vector residual block $h_{\ell+1} = h_\ell + F_\ell(h_\ell)$, derive its Jacobian and explain how the identity term can help without guaranteeing successful training. (2 points)

B CNN dimensions and parameter counts 6 points

The input shape is C × H × W = 3 × 24 × 24; omit the batch dimension. All convolutions use dilation 1, groups 1, no padding, stride 1, and one trainable bias per output channel. Each max-pooling layer uses a 2 × 2 window, stride 2, no padding, and floor rounding.



Original CNN figure from Homework3_new2.docx. Use the explicit strides and bias conventions stated above.

Layer	Specification
Conv 1	12 output channels; 3 × 3 kernel
Pool 1	2 × 2 max pooling; stride 2
Conv 2	128 output channels; 3 × 3 kernel
Pool 2	2 × 2 max pooling; stride 2

(i) Compute the output shape after every layer. Show the dimension formula and intermediate calculations. (2 points)

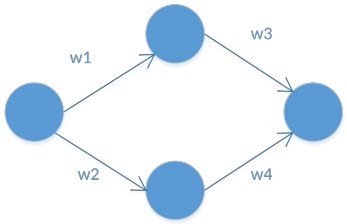
(ii) For each convolution, report weights per output filter, biases per output filter, and total trainable parameters in the layer. State the parameter count of each pooling layer. (2 points)

(iii) Compute the flattened feature-vector length after Pool 2. If it feeds a fully connected layer with 10 outputs and a bias for every output, compute the additional parameter count. Explain how convolutional weight sharing changes parameter usage compared with a fully connected mapping. (2 points)

Question 2 Backpropagation and gradient verification

10 points

For one example (x, y), where y ∈ {0, 1}, define the network below. All weights are scalars and there are no biases. Only the hidden activation ϕ changes between parts; the output is always sigmoid.



Original network: left node is input x; middle nodes are h_1, h_2; right node is output $\hat{y}$.

$$h_1 = \phi(w_1 x), \quad h_2 = \phi(w_2 x), \quad z = w_3 h_1 + w_4 h_2, \quad \hat{y} = \sigma(z).$$

$$\mathcal{L} = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y}).$$

Use natural logarithms. In code, compute the equivalent binary cross-entropy directly from logits z using a numerically stable implementation.

A Sigmoid hidden units 4 points

Set $\phi = \sigma$. Derive ∂L/∂z and ∂L/∂w$_i$ for i = 1, 2, 3, 4, showing the chain-rule factors. Write the simultaneous gradient-descent updates with learning rate η; all gradients must be evaluated at the pre-update weights.

B Smooth activation 3 points

Set ϕ(u) = log(1 + exp(2ku))/(2k), with k > 0. Derive ϕ'(u) and the four weight gradients. Explain its limiting relationship to ReLU as k increases. Explain how ReLU can be used in backpropagation despite not being differentiable at zero. This scaled Softplus is an alternative activation, not a prerequisite for training with ReLU.

C Numerical verification 3 points

Use x = 0.7, y = 1, and (w_1, w_2, w_3, w_4) = (0.2, -0.3, 0.4, -0.5). For sigmoid and for scaled Softplus with k = 1, compare your analytic gradients with automatic differentiation and central finite differences using $\varepsilon = 10^{-5}$ and float64 arithmetic. Report all four gradients and the maximum absolute discrepancy for each comparison. Explain why choosing a much smaller ε need not improve agreement.

Question 3 Evaluation protocol audit

5 points

A student standardizes the entire dataset, makes an 80/20 training/test split, trains 20 architectures, and reports the architecture with the highest test R^2. The student repeats this procedure five times and claims an unbiased estimate of generalization.

(a) Identify and explain the preprocessing leakage and test-based model-selection bias. Explain why repetition does not remove these problems. (2 points)

(b) Propose a corrected sequence for splitting, fitting preprocessing, tuning, early stopping, and final evaluation. (2 points)

(c) Suppose several rows represent measurements of the same material. Explain how the split should change if the intended use is prediction for previously unseen materials. State what identifying metadata would be needed. (1 point)

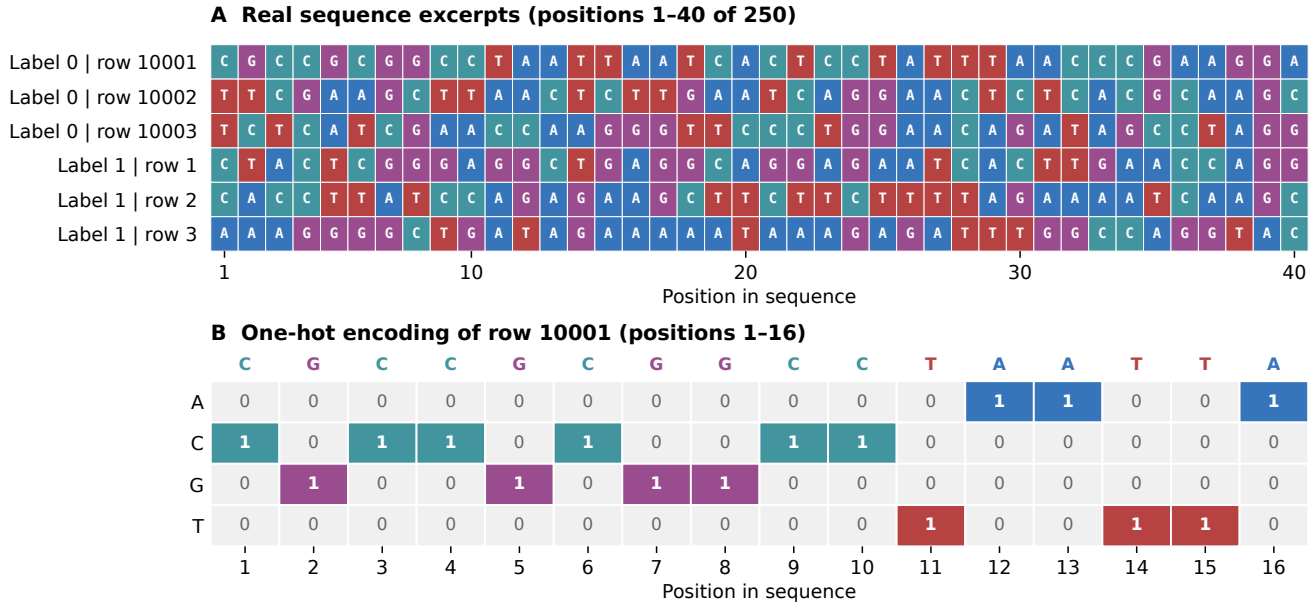
Question 4 What information distinguishes the DNA classes

15 points

AI coding is allowed, including AI generation of the complete implementation.

Research question. Does DNA sequence classification depend primarily on nucleotide composition or on sequence order? Investigate through ML baselines and controlled experiments.

The dataset contains two classes of DNA sequences, one is from human, the other from non-human. The goal is to train a classifier to differentiate them. Use `humanvsran.seq.txt` and `humanvsran_label.txt`. There are 20,000 aligned sequence/label records, each with 250 bases from A, C, G, T, and 10,000 records per numeric class. Keep sequences aligned with labels.



Dataset examples. Panel A shows positions 1–40 of the first three records in each numeric class; row numbers are 1-based positions in the supplied files. Panel B expands positions 1–16 of the first displayed record into the A, C, G, T one-hot channels. Only excerpts are shown; each full input has 250 bases. These are illustrative examples, not a representative sample or evidence of class differences. Numeric labels are preserved without assigning biological meanings.

A Hypothesis and data preparation

State your hypothesis (Does DNA sequence classification depend primarily on nucleotide composition or on sequence order?), predict the outcomes of the frequency baseline and the shuffling control, and specify what evidence would challenge your hypothesis. Verify sequence lengths, alphabet, class balance, exact duplicates, and conflicting labels. retain the prescribed row split for the main comparison.

For each seed, use stratified splits of 12,000 training, 3,000 validation, and 5,000 test records. Encode A, C, G, T in that fixed order. Use a flattened 1,000-element input for the MLP and a 4×250 input for the 1D CNN. The figure shows excerpts only; train on full sequences.

Question 4 continued

B Required baselines and control

Train and compare the results of 3 classifiers: (1) logistic regression on four nucleotide frequencies (L2 regularization, $C = 1$), (2) an MLP with two hidden layers, and (3) a 1D CNN with two convolutional layers and a classification head. Convolve along sequence positions with four nucleotide channels. Use binary cross-entropy for the neural models and threshold class-1 probabilities at 0.5. Report layer shapes, parameter counts, optimizer, learning rate, batch size, and regularization. Use common splits and the shared tuning budget.

After choices are frozen, independently shuffle positions within each test sequence, preserving its nucleotide counts and label. Use perturbation seed $s + 100$ for split seed s, with identical shuffled inputs for all models. Evaluate the frozen models without retraining on both original and shuffled test sets. Report changes in accuracy across the prescribed splits. Explain what each control can and cannot establish.

C Choose one additional investigation

Choose one option and explain why it tests your hypothesis. Use the same splits and no more than three factor settings. You need not complete every option.

1. **Neighboring-base information.** Compare logistic regression on four nucleotide frequencies with logistic regression on the same four features plus 16 dinucleotide frequencies. Normalize dinucleotide counts by the number of adjacent pairs. Keep regularization and evaluation fixed.
2. **Local pattern scale.** Compare two CNN kernel widths, such as 3 and 9. Keep depth and training rules fixed and aim for a similar parameter budget. Report any remaining capacity difference and its effect on interpretation.
3. **Information remaining after shuffling.** Retrain the chosen neural architecture on independently position-shuffled training sequences and select epochs using shuffled validation sequences. Evaluate on shuffled test sequences. Contrast this result with the frozen-model perturbation; use recorded seeds and no cross-partition mixing.
4. **Data efficiency.** Train the CNN on nested, stratified subsets containing 25%, 50%, and 100% of the training records. Keep validation and test sets fixed within each split. Plot performance against training-set size and state whether your budget holds epochs, updates, or runtime fixed.

D Evidence and conclusion

Report accuracy and balanced accuracy for each core model and chosen condition, per split and as mean ± standard deviation. Include paired CNN-minus-MLP accuracy differences, training/validation curves for the core neural models on seed 11, and a figure that answers your chosen investigative question.

Explain whether results support your hypothesis, your strongest supported conclusion, a plausible alternative explanation, and one follow-up experiment that would distinguish the explanations. High accuracy or sensitivity to shuffling alone does not establish learned biological mechanisms. Discuss the generalization limits of random row splitting and the metadata needed for a stronger genomic-region or sequence-family split.

Grading: hypothesis and predictions 3 points; valid implementation and evaluation 4 points; controlled experiments and reproducible evidence 5 points; interpretation, limitations, and next experiment 3 points. Include the AI-use statement and implementation check described on page 1.

Optional starting reference: `https://github.com/pytorch/examples/tree/main/mnist`. Adapt the training workflow to 1D sequences; MNIST is not a required experiment.

Question 5 When does a neural network help in regression

10 points

AI coding is allowed, including AI generation of the complete implementation.

Research question. We are training a transition temperature prediction problem over a superconductor dataset. We want to investigate When and why does an MLP improve, or fail to improve, over simple baselines? Supported negative results can earn full credit.

Use `superconductor.csv` only: predict the final column, `critical_temp`, from the preceding 81 features. Report target units.

A Hypothesis and common evaluation

State your hypothesis on what factors may affect your DNN model performance before training. Audit counts, missing/nonfinite values, target range, and duplicates of the dataset; Hold out 20% for testing, then 20% of development rows for validation (approximately 64/16/20 overall). Share indices across models and use the three prescribed seeds.

Fit preprocessing on training rows only and invert any target scaling before scoring. Discuss recurring related materials across partitions and why random row splits do not establish generalization to new material families.

B Baselines and one investigation

Use AI coding to build and compare three ML models: a training-target-mean predictor, ridge regression, and a base MLP with two 64-unit ReLU hidden layers and a linear output. Train the MLP with mean squared error, standardized features, no dropout, and zero weight decay. Select ridge $\alpha \in \{0.1, 1, 10, 100\}$ on each validation split. The MLP may select learning rate and batch size from at most two declared configurations per split.

Choose **one** investigation. Hold other settings fixed within each split. Report parameter counts, training budgets, and remaining confounders.

1. **Training-set size.** Use nested 25%, 50%, and 100% training subsets with fixed validation/test sets. Fit baselines and preprocessing on each matching subset. Plot the MLP's advantage over ridge against data size. State whether epochs, updates, or runtime are held fixed.
2. **Regularization.** Compare weight decay 0, 10^{-4}, and 10^{-2} using the same optimizer and architecture. Examine fitting, overfitting, and consistency across splits.
3. **Capacity.** Compare one, two, and three 64-unit ReLU layers. Does capacity improve validation performance or mainly reduce training error? Acknowledge that depth and parameter count both change.
4. **Feature scaling.** Compare standardized and unscaled inputs with the same architecture, optimizer, learning rate, batch size, and target treatment. Investigate convergence and prediction quality under these fixed optimization settings.

C Additional strategy to improve the performance?

In addition to above ideas, can you propose an additional idea or ideas to improve the performance by discussing with AI model?

E Results and error analysis

Report test R^2, MAE, and RMSE for every baseline and chosen condition, per split and as mean $\pm$ standard deviation. Use original target units for MAE/RMSE and include paired differences from the reference. Present one plot answering your research question and training/validation curves for seed 11. Explain negative R^2 values if they occur.

Before testing, define one subgroup using training data only, such as targets above the training-target median. Apply that fixed definition to the test set; report subgroup MAE and sample count for the MLP and ridge on each split. Include a base-MLP residual plot for seed 11. Does the overall score hide a region with large errors? Do not tune using this test error analysis.

E Interpretation and assessment

State whether the hypothesis and an MLP advantage are supported. Give one alternative explanation and one next experiment. Include the AI-use statement and implementation check; distinguish findings from speculation.

Grading: hypothesis and design 2 points; baselines, controls, and valid evaluation 3 points; results, variability, and error analysis 3 points; conclusions and limitations 2 points.